

QUETTA TECH SOLUTIONS

SOFTWARE REQUIREMENTS SPECIFICATION & TECHNICAL EVALUATION

Project: Full-Stack IT & Computer Services Platform
Stack: PHP 8+, MySQL (PDO), Bootstrap 5, JS
Classification: Academic Final Project & Client SRS

4 RELATIONAL TABLES	100% PREPARED STATEMENTS	Full CRUD SERVICES & GALLERY	BCRYPT PASSWORD HASHING
------------------------	-----------------------------	---------------------------------	----------------------------

1. EXECUTIVE SUMMARY & MARKET CONTEXT

This document details the Software Requirements Specification (SRS) and technical implementation for **Quetta Tech Solutions**, an enterprise-grade full-stack web application designed for computer repair, enterprise networking, and IT infrastructure services in Quetta, Balochistan. The system bridges the gap between customer inquiry handling, transparent repair pricing, and secure back-office administrative operations without recurring SaaS licensing overhead.

2. BUSINESS PROBLEM & OPERATIONAL BOTTLENECKS

2.1 Regional Industry Deficiencies in Quetta

The computer hardware and IT support ecosystem in Quetta has historically functioned via unorganized, brick-and-mortar storefronts along **Zarghoon Road, Jinnah Road, and Liaquat Bazaar**. While regional businesses, schools, and hospitals increasingly rely on IT infrastructure, the local repair sector operates under severe operational constraints:

Key Operational Inefficiencies Identified:

- Information Asymmetry & Pricing Opacity:** Customers face arbitrary repair estimates, surprise diagnostic fees, and undocumented hardware replacement pricing. Trust remains low without an authenticated public baseline pricing catalog.
- Fragmented & Lost Lead Intake:** Inquiries collected via handwritten shop slips or informal WhatsApp chats result in lost leads, unassigned customer tickets, and zero accountability.
- Absence of Verifiable Visual Portfolio:** Technical competencies (such as motherboard micro-soldering, CCTV installations, and fiber routing) cannot be verified by enterprise clients without an authenticated digital portfolio.
- Administrative Data Silos:** Small-to-medium IT businesses lack an accessible, lightweight administrative system to manage service catalogs, inspect customer inquiries, and archive completed projects.

2.2 Formal Problem Statement

"How can a localized IT engineering firm in Quetta establish a secure, high-performance, and verifiable digital platform that automates customer inquiries, transparently publishes dynamic service pricing, and provides administrative staff with authenticated CRUD management over services and portfolio records without relying on bloated, high-overhead frameworks?"

2.3 System Boundaries (SRS Scope)

The platform provides: (1) A responsive, high-speed public web portal with dynamic service catalog and inquiry processing, and (2) A protected administrative console with real-time analytics, full CRUD operations for services and gallery assets, and message inbox management.

3. PROPOSED TECHNICAL SOLUTION & ARCHITECTURE

The system adheres to an **MVC-separated architecture** utilizing native PHP 8+ PDO for data access and modern browser standards for presentation, ensuring maximum security and performance.

3.1 Layered System Design

Layer / Tier	Technologies & Components	Architectural Responsibility
Presentation Layer	HTML5, CSS3, Bootstrap 5.3, Vanilla JS, Bootstrap Icons	Responsive UI rendering across 320px–1440px breakpoints; dynamic counter animations; instant file upload previewing.
Controller / Logic	Native PHP 8+, Session Middleware, Helper Utilities	Input sanitization via <code>sanitize()</code> , CSRF validation, HTTP redirect handling, flash message queueing.
Security & Auth	<code>password_hash(PASSWORD_BCRYPT)</code> , <code>password_verify()</code>	Constant-time password verification, session fixation defense via <code>session_regenerate_id(true)</code> , Route guards.
Data Access (PDO)	Singleton PDO Factory, UTF-8 Prepared Statements	Parameterized query execution preventing 100% of SQL injection vectors; native data type binding.
Persistence Layer	MySQL InnoDB Relational Engine	Relational integrity, foreign key cascades, unique indexing on usernames/emails.

3.2 Relational Database Schema Design

The persistence layer consists of 4 normalized, relational tables in the `quetta_tech_solutions` database:

- `users` (`id` PK, `username` UNIQUE, `email` UNIQUE, `password`, `created_at`) — Administrative staff credentials.
- `services` (`id` PK, `user_id` FK $\rightarrow$ `users.id` ON DELETE CASCADE, `title`, `description`, `price`, `image`, `created_at`) — Active IT service catalog.
- `gallery` (`id` PK, `service_id` FK $\rightarrow$ `services.id` ON DELETE SET NULL, `image`, `caption`, `created_at`) — Visual project proof.
- `contact_messages` (`id` PK, `name`, `email`, `phone`, `subject`, `message`, `created_at`) — Inbound customer inquiry tickets.

3.3 Core Modules & Admin Subsystem

The administration subsystem provides authenticated staff with real-time KPI metrics (Total Services, Total Gallery Images, Total Inquiries) and full CRUD capabilities for services and gallery records with automatic disk file synchronization (`unlink()`) upon record updates and deletions.

QUETTA TECH SOLUTIONS

SECTION 4 & 5: AI TOOLS, CHALLENGES FACED & CONCLUSION

4. AI TOOLS USED & ENGINEERING METHODOLOGY

Advanced AI coding assistants (**Google Antigravity IDE with Gemini 3.7**) were strategically utilized throughout the development lifecycle as an architectural pair-programmer across four key domains:

Phase	AI Workflow & Prompting	Implemented Deliverable
1. Database DDL	Schema normalization prompts specifying InnoDB foreign keys and BCrypt seed hashes.	Production-ready <code>quetta_tech_solutions.sql</code> with constraints and sample services.
2. Security Stack	Defense-in-depth prompting for image uploads (finfo magic bytes) and CSRF tokens.	Hardened helper library in <code>includes/functions.php</code> and <code>includes/auth.php</code> .
3. Automated Testing	Subagent execution for unit test scripts and headless browser navigation.	19 automated PHP tests in <code>test_suite.php</code> and end-to-end browser validation.
4. Frontend System	Tailored tokens for modern tech aesthetic (Navy <code>#0b132b</code> , Cyan <code>#00b4d8</code>).	Responsive styling in <code>assets/css/style.css</code> and <code>admin.css</code> .

5. ENGINEERING CHALLENGES FACED & SOLUTIONS

1. PHP "Headers Already Sent" on Form Redirection:

Challenge: Initial implementations included HTML header view templates before processing POST requests, causing `header('Location: ...')` redirects to fail.

Resolution: Re-architected all form controllers (`contact.php`, `admin/services/create.php`, etc.) so that authentication, CSRF validation, file uploads, and database writes execute completely *before* rendering HTML headers.

2. Malicious File Upload Prevention (PHP Shells):

Challenge: Standard `$_FILES` array variables can be spoofed by renaming malicious PHP scripts to image extensions.

Resolution: Built a 4-tier pipeline: extension whitelisting, true binary MIME inspection using PHP's `finfo(FILEINFO_MIME_TYPE)`, 5MB size limit, and randomized cryptographically unique filenames.

3. Disk Orphan File Cleanup:

Challenge: Updating or deleting database records left old image files occupying storage space.

Resolution: Integrated `delete_uploaded_file()` hooks using PHP's `unlink()` to automatically remove old files on replacement or record deletion.

4. Session Fixation & CSRF Mitigation:

Challenge: Unprotected admin endpoints could be hijacked via forged requests or session fixation.

Resolution: Embedded synchronized CSRF tokens in all state-changing forms and enforced `session_regenerate_id(true)` on login.

6. CONCLUSION & FUTURE ROADMAP

The **Quetta Tech Solutions** project delivers a production-ready full-stack web application. Future iterations will incorporate: (1) SMS/WhatsApp automated repair alerts, (2) Online customer ticket tracking portal, and (3) Multi-role RBAC for staff and technicians.